

QuickPass

Technical Documentation

Cryptographically Rotating QR Attendance System

Event: Prasunethon 2.0 Hackathon

Author: Lokesh Saini

Email: lksaini2804@gmail.com

Live Demo: <https://quick-pass-client-bl4w.vercel.app>

API Health: <https://quickpass-backend-cv0o.onrender.com/health>

Date: August 2026

One-line pitch: QuickPass treats classroom attendance as a short-lived cryptographic token – not a static image – eliminating both proxy fraud and the time overhead of manual roll calls.

Contents

1	Overview	3
1.1	Problem Statement	3
1.2	Solution	3
1.3	Security Pillars	3
2	System Architecture	4
2.1	High-Level Overview	4
2.2	Repository Layout	4
2.3	Request Lifecycle	5
3	Data Models	6
3.1	Entity Relationships	6
3.2	Schema Definitions	6
3.2.1	User	6
3.2.2	Classroom	6
3.2.3	Session	6
3.2.4	AttendanceLog	6
4	Backend API Reference	7
4.1	Authentication – /api/auth (Public)	7
4.2	Teacher API – /api/teacher (Bearer JWT, role=teacher)	7
4.3	Student API – /api/student (Bearer JWT, role=student)	7
4.4	Utility Endpoint	7
5	Core Security Mechanisms	8
5.1	QR Payload Cryptography	8
5.2	Scan Validation Pipeline	9
5.3	Anti-Proxy Device Fingerprinting	9
5.4	WebSocket Event Reference	10
6	Frontend Architecture	11
6.1	Technology Stack	11
6.2	QR Scanner Phase State Machine	11
6.3	Backend Wake-up Mechanism	11
7	Deployment	12
7.1	Environment Variables	12

7.2	Production URLs	12
7.3	Local Development	12
8	Analytics and Reporting	13
8.1	Attendance Analytics Endpoint	13
8.2	Low-Attendance Email Alerts	13
8.3	Client-Side CSV Export	13
9	Known Limitations	14
10	Glossary	15

1 Overview

1.1 Problem Statement

Classroom attendance management fails in two specific ways:

1. **Time overhead** – Manual roll calls and static QR check-ins consume 5–10 minutes per lecture, compounding across an academic year.
2. **Proxy fraud** – A screenshot of a static QR code shared over WhatsApp, or a single phone passed around a room, renders attendance data meaningless.

1.2 Solution

QuickPass makes attendance marks *cryptographically non-transferable*:

- The server encrypts `{sessionId, teacherId, timestamp}` with **AES-256-CBC** and broadcasts the ciphertext to the teacher's display via WebSocket every **10 seconds**.
- A student scans the live code; the encrypted payload travels to the server.
- The server decrypts, validates age (≤ 10 s), session state, enrollment, and device binding before accepting the mark.
- Screenshots die because the embedded timestamp expires before they can be re-used.

1.3 Security Pillars

Table 1: Core security mechanisms

Pillar	What happens	Implementation
Dynamic payload	JSON encrypted and rotated every 10s	<code>server/utils/crypto.js</code>
Expiry check	Reject tokens older than 10 000 ms	<code>validateQRPayload()</code>
Device binding	Browser fingerprint sent on each scan; reuse flags both students	<code>client/src/utils/helpers.js</code>
Role-based access	Separate teacher/student JWT scopes	<code>middleware/auth.js</code>

2 System Architecture

2.1 High-Level Overview

QuickPass is a two-tier web application deployed across two cloud platforms:

- **Frontend** – React 19 SPA hosted on **Vercel** (CDN-distributed).
- **Backend** – Node.js + Express + Socket.io hosted on **Render**.
- **Database** – MongoDB Atlas (cloud-hosted NoSQL).
- **Email** – Resend API with verified domain `qrmanager.me`.

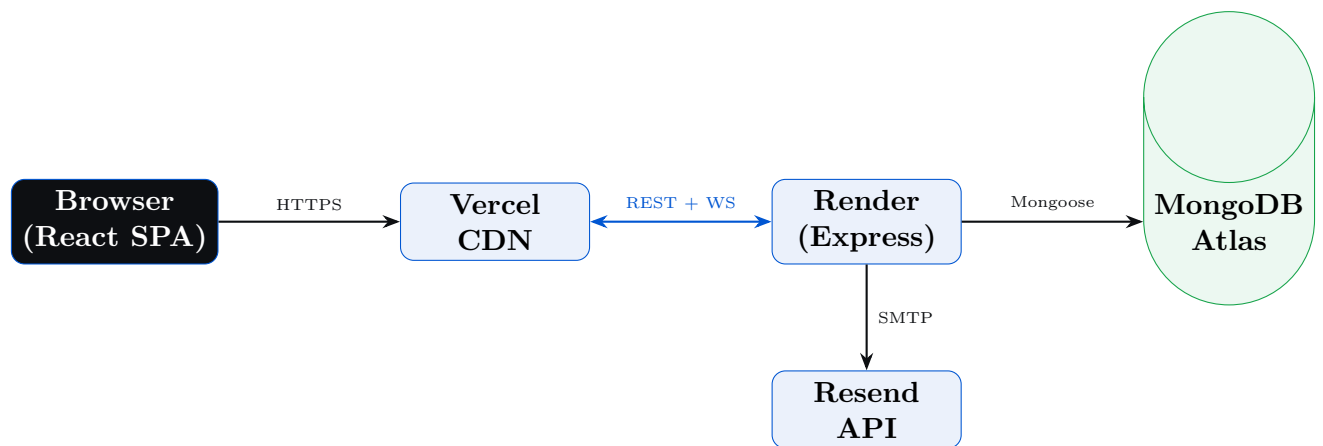


Figure 1: Production deployment topology

2.2 Repository Layout

```

quickpass/
|-- package.json           # Root workspace scripts
|-- client/                # React 19 + Vite (port 5173)
|   '-- src/
|       |-- App.jsx        # Router + backend health-ping
|       |                   on load
|       |-- api/
|       |   |-- axios.js    # Axios instance with JWT
|       |   |               interceptor
|       |   '-- socket.js   # Socket.io singleton (JWT
|       |                   handshake)
|       |-- context/AuthContext.jsx
|       |-- pages/
|       |   |-- LandingPage.jsx
|       |   |-- LoginPage.jsx / RegisterPage.jsx

```

```

|         | |-- teacher/      # Layout, Overview, Classrooms,
|         |         Sessions, Analytics
|         | '-- student/      # Layout, Overview,
|         |         QRScannerPage, Attendance, Notes, Schedule
|         '-- utils/helpers.js # getDeviceId(), date formatters
'-- server/                    # Express + Socket.io (port
    5000)
    |-- index.js               # HTTP server + QR interval
    |         engine
    |-- routes/
    |   |-- auth.js
    |   |-- teacher.js
    |   '-- student.js
    |-- models/
    |   |-- User.js   Classroom.js   Session.js   AttendanceLog.
    |   |         js   Note.js
    |-- middleware/
    |   |-- auth.js           # protect() + authorize(role)
    |   '-- errorHandler.js
    '-- utils/
        |-- crypto.js         # AES-256-CBC encrypt / decrypt
        |-- jwt.js
        '-- email.js          # Resend API wrapper

```

2.3 Request Lifecycle

1. **REST (state changes):** Login issues JWT → Axios attaches Authorization: Bearer → protect() verifies signature → authorize(role) enforces RBAC → route handler → MongoDB.
2. **WebSocket (real-time session):** Teacher emits join_session → server starts a 10s setInterval generating new QR payloads → broadcasts qr_refresh to the session room. Successful scans emit attendance_marked; device conflicts emit proxy_flagged.
3. **Email (async notification):** Teacher triggers POST /notify-low-attendance → backend finds enrolled students below 75 % → Resend API delivers email from admin@qrmanager.me.

3 Data Models

3.1 Entity Relationships

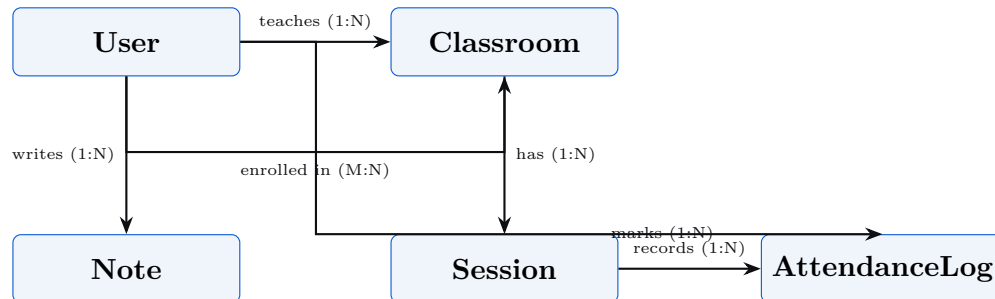


Figure 2: MongoDB collection relationships

3.2 Schema Definitions

3.2.1 User

Field	Type	Req	Notes
name	String	Yes	Full display name
email	String	Yes	Unique index, lowercase
password	String	Yes	bcrypt cost 10; stripped in toJSON()
role	String	Yes	"teacher" or "student"
studentId	String	No	Optional institutional roll number

3.2.2 Classroom

Field	Type	Req	Notes
teacherId	ObjectId	Yes	Ref: User
name	String	Yes	Course full name
courseCode	String	Yes	Uppercase trimmed
schedule	Array	No	[{day, startTime, endTime}]
enrolledStudents	Array	–	ObjectId refs to User

3.2.3 Session

3.2.4 AttendanceLog

Field	Type	Req	Notes
classroomId	ObjectId	Yes	Ref: Classroom
date	Date	Yes	Session date
startTime	Date	Yes	When the session opened
endTime	Date	No	Set when session ends
status	String	Yes	"active" or "completed"

Field	Type	Req	Notes
sessionId	ObjectId	Yes	Ref: Session
studentId	ObjectId	Yes	Ref: User
deviceId	String	Yes	Browser fingerprint hash
timestamp	Date	Yes	Server-set scan time
status	String	Yes	present, flagged_proxy, or cancelled
undoExpiresAt	Date	No	timestamp + 5min
proxyFlag	Object	No	{suspectedDeviceId, suspectedStudentId}

Table 2: Compound unique index on (sessionId, studentId)

4 Backend API Reference

4.1 Authentication – /api/auth (Public)

Method	Path	Body	Response
POST	/register	name, email, password, role, [studentId]	201 {token, user}
POST	/login	email, password	200 {token, user}

JWT payload: {id, role}, signed with JWT_SECRET, 7-day expiry. bcryptjs cost factor: 10. Duplicate email ⇒ 409. Wrong credentials ⇒ 401.

4.2 Teacher API – /api/teacher (Bearer JWT, role=teacher)

4.3 Student API – /api/student (Bearer JWT, role=student)

4.4 Utility Endpoint

GET /health – Returns {status: "ok", timestamp}. Automatically pinged by the frontend on every page load to pre-warm Render's free-tier instance.

Method	Path	Purpose
GET	/classrooms	List teacher's classrooms
POST	/classrooms	Create a classroom
PUT	/classrooms/:id	Update classroom
DELETE	/classrooms/:id	Delete classroom
POST	/classrooms/:id/enroll	Enroll student by email or studentId
DELETE	/classrooms/:id/enroll/:studentId	Remove a student
GET	/classrooms/:id/sessions	Session history
POST	/classrooms/:id/sessions/start	Open a new active session
POST	/sessions/:sessionId/end	Close the active session
GET	/sessions/:sessionId/attendance	Live attendance roster
GET	/classrooms/:id/analytics	Per-student stats and proxy flags
POST	/classrooms/:id/notify-low-attendance	Alert students below 75 % attendance
GET	/sessions/:sessionId/qr	REST QR fallback (if WebSocket fails)

Method	Path	Purpose
POST	/scan	Decrypt QR, validate, mark attendance
POST	/undo/:logId	Cancel attendance within 5 minutes
GET	/my-classrooms	Enrolled classrooms
GET	/my-attendance	Per-class stats and history
GET	/active-sessions	Currently running sessions
GET	/schedule	Weekly schedule view
GET	/notes/:classroomId	List notes for a class
POST	/notes/:classroomId	Create a note
PUT	/notes/:noteId	Update own note
DELETE	/notes/:noteId	Delete own note

5 Core Security Mechanisms

5.1 QR Payload Cryptography

File: server/utils/crypto.js

Algorithm: AES-256-CBC **Key:** 32-byte env variable AES_ENCRYPTION_KEY
IV: 16 cryptographically random bytes, unique per payload.
Wire format: ivHex:ciphertextHex (colon-delimited hex strings).

Plaintext JSON before encryption:

```
1 {
2   sessionId: "<MongoDB ObjectId>",
```

```

3   teacherId: "<MongoDB ObjectId>",
4   timestamp: Date.now()    // milliseconds since epoch (
      server clock)
5 }

```

Listing 1: QR payload plaintext structure

Validation in validateQRPayload():

1. Split on ":" to extract IV hex and ciphertext hex.
2. AES-256-CBC decrypt using the server key and extracted IV.
3. `JSON.parse()` the decrypted string.
4. Assert `sessionId`, `teacherId`, `timestamp` are present.
5. Compute `age = Date.now() - timestamp`. Reject if `age > 10000` or `age < 0`.

Students never receive the AES key – they only read the ciphertext with a camera. Screenshot reuse *always* fails: the embedded timestamp is stale (>10s) before the screenshot can reach the server.

5.2 Scan Validation Pipeline

Every POST `/api/student/scan` executes these checks sequentially:

1. **Decrypt** – `validateQRPayload(encryptedPayload) ⇒ 400` on failure.
2. **Session state** – Session must be `active` ⇒ 400.
3. **Enrollment** – Student must be in `Classroom.enrolledStudents` ⇒ 403.
4. **Duplicate check** – Existing `present` log for same (session, student) ⇒ 409.
5. **Device binding:**
 - `deviceId` already used by *another* student in this session ⇒ create `flagged_proxy` log, downgrade original to `flagged_proxy`, emit `proxy_flagged` via Socket.io, return 403.
 - Otherwise ⇒ create `present` log (`undoExpiresAt = now + 5min`), emit `attendance_marked`, return 201.

5.3 Anti-Proxy Device Fingerprinting

File: `client/src/utils/helpers.js`

```

1 const getDeviceId = () => {
2   // Stable across sessions for the same browser profile
3   if (localStorage.getItem('qp_device_id'))
4     return localStorage.getItem('qp_device_id');
5
6   const raw = [

```

```

7     navigator.userAgent, navigator.language, screen.
        colorDepth,
8     screen.width, screen.height, new Date().getTimezoneOffset
        (),
9     navigator.hardwareConcurrency, navigator.platform
10 ].join('|');
11
12 // djb2-style hash + random suffix stored in localStorage
13 const id = djb2Hash(raw) + '-' + Math.random().toString(36)
        .slice(2, 8);
14 localStorage.setItem('qp_device_id', id);
15 return id;
16 };

```

Listing 2: Client-side device ID generation

5.4 WebSocket Event Reference

Table 3: Socket.io events for the session room `session_{sessionId}`

Event	Dir.	Role	Effect
join_session	C → S	teacher	Join room; start 10 s QR interval
leave_session	C → S	teacher	Leave room; clear interval if empty
join_session_as_student	C → S	student	Join room for live updates
qr_refresh	S → C	–	{payload, timestamp}
attendance_marked	S → C	–	Populated AttendanceLog document
proxy_flagged	S → C	–	Conflict details
attendance_cancelled	S → C	–	After student undo

The QR interval map `Map<sessionId, intervalId>` in `server/index.js` ensures only one `setInterval` runs per session regardless of how many teacher connections join.

6 Frontend Architecture

6.1 Technology Stack

Concern	Technology
Framework	React 19 + Vite 8
Routing	React Router v7 (nested layout routes)
Styling	Tailwind CSS 3 + custom design tokens (Source Sans Pro)
State	React Context API (AuthContext) – no Redux
HTTP	Axios with JWT interceptor + automatic 401 redirect
WebSocket	Socket.io-client (singleton, JWT handshake on init)
QR Generation	qrcode npm package
QR Scanning	html5-qrcode with phase-state machine
Email (server)	@resend/node SDK, domain <code>qrmanager.me</code>
Markdown notes	react-markdown
Toasts	react-hot-toast

6.2 QR Scanner Phase State Machine

The `QRScannerPage.jsx` uses a five-phase state machine to fix a critical DOM timing bug: `Html5Qrcode.start()` was previously called before React had flushed the target `<div>` to the DOM, causing a “Failed to start scanner” error on mobile.

Phase	What happens
idle	“Start Camera” button rendered; no camera access yet.
requesting	<code>navigator.mediaDevices.getUserMedia()</code> called – fires the browser permission prompt.
scanning	Scanner <code><div></code> rendered in DOM; <code>useEffect</code> fires after 150 ms, then <code>Html5Qrcode.start()</code> is called safely.
success	Attendance marked; 5-minute undo countdown shown.
error	Specific error displayed (<code>NotAllowedError</code> vs other); retry offered unless permission permanently denied.

Table 4: QR scanner phase state machine

6.3 Backend Wake-up Mechanism

Render’s free tier spins down instances after 15 minutes of inactivity. `App.jsx` fires a silent `GET /health` on every page load, before the user reaches any interactive element. This pre-warms the backend during the user’s natural reading time on the landing page.

7 Deployment

7.1 Environment Variables

Variable	Req.	Description
MONGO_URI	Yes	MongoDB Atlas connection string
JWT_SECRET	Yes	Signing secret for 7-day session tokens
AES_ENCRYPTION_KEY	Yes	Exactly 32 characters – AES-256 symmetric key
PORT	No	HTTP listen port; default 5000
CLIENT_URL	No	CORS + Socket.io origin; default <code>http://localhost:5173</code>
RESEND_API_KEY	No	Resend API key for email delivery
EMAIL_FROM	No	Sender address on verified Resend domain

The server hard-exits on startup if any of the first three variables are missing, or if the AES key is not exactly 32 bytes.

7.2 Production URLs

- **Frontend:** <https://quick-pass-client-bl4w.vercel.app>
- **Backend:** <https://quickpass-backend-cv0o.onrender.com>
- **Health:** <https://quickpass-backend-cv0o.onrender.com/health>

7.3 Local Development

```

npm run install:all          # install client + server
                             dependencies
cp server/.env.example server/.env
# fill in MONGO_URI, JWT_SECRET, AES_ENCRYPTION_KEY (32 chars
)

npm run seed                 # wipe DB and create demo users/
                             classrooms

npm run dev:server           # http://localhost:5000
npm run dev:client           # http://localhost:5173 (Vite
                             proxies /api)

```

Seed credentials: `teacher@quickpass.dev` / `student1@quickpass.dev` – password: `password123`

8 Analytics and Reporting

8.1 Attendance Analytics Endpoint

GET `/api/teacher/classrooms/:id/analytics` returns:

- **Per-student stats:** total sessions, present count, attendance percentage, proxy flag count.
- **Proxy flag details:** full list of `flagged_proxy` logs with suspected device and student IDs.
- **Session history:** chronological list with start/end times.

Attendance percentage formula:

$$\text{Attendance}\% = \frac{\text{presentCount}}{\text{totalSessions}} \times 100$$

8.2 Low-Attendance Email Alerts

When a teacher clicks *Notify Low-Attendance Students*:

1. Backend computes attendance percentage for each enrolled student.
2. Filters students where $\text{Attendance}\% < 75$.
3. Sends a styled HTML email via Resend API from `admin@qrmanager.me`.
4. Email body includes the student name, course, current percentage, and login link.

8.3 Client-Side CSV Export

Teachers export the current analytics view directly from the browser (no server round-trip):

- Generates a Blob with MIME type `text/csv;charset=utf-8`.
- **Columns:** Student Name, Student Email, Student ID, Present Count, Total Sessions, Attendance %, Proxy Flags.
- Downloaded as `Attendance_Report_<ClassName>.csv` via a programmatically triggered hidden anchor tag.

9 Known Limitations

Limitation	Details
Browser-scoped device ID	Clearing site data or using a second browser generates a fresh ID. True hardware-level binding would require TPM attestation or WebAuthn.
Free-tier cold starts	Render spins down after 15 min inactivity; the health-ping mitigates but cannot eliminate the startup window.
Clock independence	Both payload generation and validation use server <code>Date.now()</code> , so client clock drift does not affect the 10 s window.
AES key management	Key is stored as an environment variable. At production scale, an HSM or cloud KMS (e.g., AWS KMS) would be more appropriate.
Email deliverability	Free SMTP is commonly filtered by institutional mail servers. The Resend + custom domain setup (<code>qrmanager.me</code>) largely resolves this.

Table 5: Honest limitations for hackathon evaluation

10 Glossary

Term	Definition
Dynamic QR	Encrypted payload rotated every 10s; not a static image.
QR payload	<code>ivHex:ciphertextHex</code> wrapping <code>{sessionId, teacherId, timestamp}</code> .
Device ID	Browser fingerprint persisted as <code>qp_device_id</code> in <code>localStorage</code> .
Proxy flag	Same <code>deviceId</code> used by two distinct students in one session.
Undo period	5 minutes post-scan during which a student may cancel their mark.
Low attendance	present count/session count < 75 %.
Socket room	Isolated WebSocket broadcast group: <code>session_{sessionId}</code> .
JWT	7-day <code>{id, role}</code> token used on HTTP and Socket.io handshake.
AES-256-CBC	Symmetric block cipher with 256-bit key and cipher block chaining.
IV	16 random bytes prepended to ciphertext; unique per payload.
RBAC	Role-Based Access Control: teacher vs. student, enforced server-side.
Phase state machine	Pattern in <code>QRScannerPage</code> sequencing permission, DOM render, and camera init.

Table 6: Glossary of terms

QuickPass · Prasunethon 2.0 · Lokesh Saini

<https://quick-pass-client-bl4w.vercel.app>

Built with React 19 · Node.js · MongoDB · Socket.io · AES-256